How to measure, analyse and close gender pay gaps

A step-by-step guide to mastering the article 9 and 10 requirements of the EU Pay Transparency Directive

0. Foreword

The Pay Transparency Directive makes transparency and accountability more important than ever—not only to mitigate legal risk, but to ensure men and women are paid fairly for their work. It's the right thing to do, and it's good business.

When we first learned about the directive at Fairfox, we immediately recognized the challenge many organizations would face. The requirements are genuinely rigorous: Article 9 mandates identifying and addressing pay gaps of 5% or more, while Article 18 reverses the burden of proof, requiring organizations to demonstrate compliance through statistical analysis rather than good intentions. These aren't tasks you can handle in a spreadsheet. That is why we built Fairfox.

As data scientists at heart, we believe in transparency. While we're thrilled when organizations choose our platform, we also respect those who want to tackle this challenge themselves. That's why we've created this guide: to give you the knowledge and tools you need to conduct rigorous pay equity analysis on your own terms.

Whether you ultimately work with us or go it alone, what matters most is that you succeed in creating fair, transparent compensation practices.

On behalf of the Fairfox team,

Phillip Kyle
Mathias Stokholm

1. Introduction

The goal of this document is to provide HR practitioners and data analysts with a practical, step-by-step methodology for analyzing your organization's gender pay gap in accordance article 9 and 10 in the pay transparency directive.

What do you need to do?

The core enforcement mechanism are articles 9 and 10.

a) report gender pay gaps "between workers performing the same work or work of equal value" to a monitoring body (article 9.1.g) ...

b) ... and to close gender pay gaps > 5% that *cannot be justified using objective, gender-neutral criteria* in a timely manner (article 10).

Failure to do above triggers a mandatory 'Joint Pay Assessment' – a comprehensive salary review conducted in collaboration with employee representatives that charts a plan for closing the unexplained pay gaps.

Who is in scope?

All EU Member States must transpose the directive into national law by June 7 with reporting requirement beginning shortly thereafter:

- **>250+ workers:** Annual reporting from June 7, 2027
- **150-249 workers:** Every three years from June 7, 2027
- **100-149 workers:** Every three years starting June 7, 2031

What if you don't

If you aren't deterred by the prospect of an expensive and tedious process like the joint pay assessment, penalties for non-compliance include fines, public shaming and possible exclusion from public tenders.

Other requirements

The directive mandates several other requirements like publishing pay ranges, prohibiting employers from asking about pay history, and making certain information is accessible. As this document serves as an analytical guide, these requirements are out of scope beyond mentioning they exist.

Who is this guide for

The purpose of this document is to equip the undeterred analyst with the tools and knowledge necessary to analyzing your pay equity. We will go through practical examples, and you can download both the code and example data used in this doc.

Download the data and code for this guide:
https://github.com/FairFox-Ai/fairfox-guide

If you are unfamiliar with regression, we **strongly recommend** that you seek help defining your pay model, as getting this wrong can have a pretty big impact in terms of both cost and legal risk.

2. What are pay gaps

In the context of the directive, we will be dealing with two different types of pay gaps – namely, unadjusted and adjusted pay gap s.

Unadjusted pay gaps

This is the simple difference between average male and female pay, expressed as a percentage of male pay:

Unadjusted Gap = (Avg. Male Pay – Avg. Female Pay) / Avg. Male Pay * 100

This is what Article 9 requires you to report (for base pay, bonuses, allowances, etc.). It is easy to calculate but can be misleading on its own — gaps can often arise because men and women tend to work in different roles, grades, or have different average tenure or performance.

To understand whether a gap is due to gender, we need to estimate the effect of gender as well as other factors that may be driving pay in the organization. While the directive does not require you to do this, we need to consider article 10 and article 18 of the directive to understand why.

Justifying pay gaps

If you report an unadjusted pay gap >5% under article 9, article 10 gives you 6 months to either objectively justify or close the gap.

Conventionally, the employer could say: this gap is not significant, thus we cannot be certain that there is a gap. This would leave it up to the claimant to prove discrimination. In this case any uncertainty would 'benefit' the employer. Article 18 reverses the burden of proof, meaning that in a case of claimed discrimination, it is up to the employer to prove that the claimant was not subject to pay discrimination.

This means that **the 'statistical uncertainty' argument won't work** and that any pay difference that an employer can't objectively justify using legitimate pay factors will be interpreted as bias.

Therefore, to demonstrate fair pay practices and meet potential article 10 requirements, we must adjust for legitimate factors (such as role, experience, and performance) that explain pay differences—this is where the adjusted pay gap becomes essential.

Adjusted pay gap

The adjusted pay gap uses regression modelling to isolate the effect of gender on pay while controlling for legitimate factors such as grade, tenure, performance, and role. This allows us to separate the contribution of each factor and identify any remaining difference attributable to gender alone.

This approach provides a rigorous, transparent, and reproducible method for objectively justifying pay differences under article 10 – particularly if your pay factors are anchored in a well-defined, publicly available pay policy.

The following equation models pay as a function of gender, grade and seniority.

$$\textbf{Pay} = \beta_0 + \beta_1 \text{Female} + \beta_2 \text{Grade} + \beta_3 \text{Seniority} + \varepsilon$$

Let's do a calculated example:

Example: Calculating the adjusted gap (%)

Step 1: Run a regression

Suppose we get these coefficients:

Coefficient	Value
β_0 (Intercept)	€30,000
β_1 (Female)	-€5,000
β_2 (Grade)	€3000 per level
β_3 (Seniority)	€2000 per year

And the mean values in our dataset are:

- Mean: Grade = 5
- Mean: Seniority = 2

Step 2: Calculate predicted average male pay

Avg. Male Pay = $\beta_0 + \beta_2$(Mean Grade) + β_3(Mean Seniority)

Avg. Male Pay = €30,000 + €3,000(5) + €2,000(2) = €49,000

Step 3: Calculate adjusted gender pay gap as %

Adj. Pay Gap (%) = [β_1 / Avg. Male Pay] × 100

Adj. Pay Gap (%) = [€5,000 / €49,000] × 100 = 10.2%

Interpretation: Women earn 10.2% less than men across the company after adjusting for grade and seniority.

Sample size

The directive requires us to justify gaps in **"...groups of workers performing the same work or work of equal value."** We could run a separate regression for each of these groups, but many groups may not be sufficiently large or have an even enough gender distribution to support a reliable regression.

Inadequate sample sizes and skewed gender distributions can cause coefficients (and gap estimates) to swing wildly in response to small changes in the data. Pay gaps you thought you closed can re-emerge with the addition or removal of a single observation.

Getting the sample size right is incredibly important, as inaccurate pay models can lead to significant costs and legal risks.

Let's explore a practical approach for determining minimum sample size requirements.

Rule of thumb: Min. 10 observations per parameter

You should have min. 10 observations per estimated parameter. The number of estimated parameters is k-1, where k is the number of levels. Continuous variables count as 1.

Example 1: Company-level analysis
Gender (2 levels): 2-1 = 1 parameter
Seniority in years: 1 parameter
Grade (7 levels): 7-1 = 6 parameters

Sample size required: (1+1+6) x 10 = 80

Example 2: Group-level analysis
When analyzing a single job group where all employees have the same grade, we exclude Grade from the model

Gender (2 levels): 2-1 = 1 parameter
Seniority in years: 1 parameter

Sample size required: (1+1) x 10 = 20

The more pay factors you include in your model (e.g., job family, location, performance), the larger sample size you'll need. Only include factors that genuinely drive pay decisions in your organization

Adjusted group gaps

When measuring adj. pay gaps across groups, we can use an interaction term to test whether the effect of one variable depends on the level of another variables. In pay gap analysis, this interaction asks: "Does the gender pay gap differ across grade levels?"

Pay model with interaction term.

$$\textbf{Pay} = \beta_0 + \beta_1(\textbf{Female}) + \beta_2(\textbf{Grade}) + \beta_3(\textbf{Female} \times \textbf{Grade}) + \beta_4(\textbf{Seniority}) + \varepsilon$$

Let's do a calculated example

Example 2: Calculating the group adj. gaps

Step1: Run a regression with interaction terms

Suppose we get these coefficients:

Coefficient	Value
β_0 (Intercept)	€30,000
β_1 (Female)	- €2,500
β_2 (Grade: 3)	€8,000
β_3 (Female × Grade 3)	€-1,500
β4 (Seniority)	€500 per year

And the mean values in our dataset are:

- Mean Seniority = 40

Step 2: Calculate the gender gap for grade 3

Gender Gap for Grade 3 = $\beta_1 + \beta_3$

-€2,500 + (-€1,500) = -<u>€4,000</u>

Step 3: Calculate average male pay for grade 3

Avg. Male Pay = $\beta_0 + \beta_2 + \beta_4$(Mean Seniority)

Grade 3: €30,000 + €8,000 + €500(40) = <u>€58,000</u>

Step 4: Convert to percentages

Gap (%) = [Gender Gap / Avg. Male Pay] × 100

Grade 3: €4,000 / €58,000 = 6.9%

By pooling our data across groups, we can better estimate pay gaps in small or skewed groups by borrowing power from the whole. This gives us both greater flexibility and more robust models.

Other considerations

It is beyond the scope of this paper to give a full introduction to multiple regression. The are many great guides on the net – for example, check out Multiple Regression for Beginners by Greb Martin.

Here are some 3 important things to consider when building your model.

1. **Include relevant pay drivers**: If you exclude important pay-determining variables (e.g., performance ratings, tenure, education), the gender coefficient may absorb their effects. This leads to biased gap estimates; you might attribute pay differences to gender when they're actually due to missing variables.

2. **Log-transform your pay variable**
   Pay distributions are typically right-skewed (few very high earners). Log transformation normalizes this and reduces the influence of extreme values without arbitrary outlier removal. Also log estimates are closer to percentages, which is how we tend to interpret pay gaps.

3. **Check for multicollinearity:** High correlation between predictors (e.g., seniority and age) inflates standard errors and makes coefficients unstable. This makes it difficult to isolate the true gender effect and produces unreliable gap estimates.

3. Step-by-Step-Guides

Before You Begin

Are you ready?

- **Minimum 100 employees** for company-level analysis
- **Minimum 200 employees** for reliable grade-level analysis
- **Job grading system in place** (Article 4(4) requires objective criteria)
- **Someone with basic data experience** to run the analysis

Gather Your Data

Required data for each employee:

- Gender (Article 9 requires sex-disaggregated data)
- Job grade or level
- Base salary
- Variable compensation (bonuses, commissions)
- Total pay
- Working hours / FTE status

Recommended control variables:

- Tenure (years with company)
- Performance rating
- Job family or function

Critical: Data Privacy (Article 12)

- Remove all personally identifiable information (names, employee IDs)
- Protect small groups that could identify individuals
- Store data on encrypted, secure systems only

4. Measuring Pay Gaps

4.1 Measuring pay gaps using R

If you haven't got R installed already, the first step is to download and install R and Rstudio

- **Download R:** https://cran.r-project.org/
- **Download RStudio:** https://posit.co/download/rstudio-desktop/

Step 1: Install and load packages

If you haven't got the packages needed from this analysis, the first step will be to install and load them.

```
# Install packages if needed (run once)
install.packages('tidyverse')
install.packages('broom')
install.packages('car')

# Load packages
library(tidyverse)
library(broom)
library(car)
```

Step 2: Load Data

The first step is to load in our dataset. A good practice is to visually review your data to make sure the data conforms to our expectations and there are no missing values.

```
# Load employee data
df <- read.csv('eu_company_pay_data.csv')

# View first few rows
head(df)

# Check structure
str(df)
```

Step 3: Prepare Variables

Before you start your pay equity analysis, make sure your data is adjusted for FTE, your compensation variable is log-transformed and the reference gender is set to male.

```
# Calculate full-time equivalent total pay
df$total_pay_fte <- df$total_pay / df$fte

# Create log-transformed pay (for percentage interpretation)
df$log_pay <- log(df$total_pay_fte)

# Set Male as reference level for gender
df$gender <- relevel(factor(df$gender), ref = "Male")
```

Step 4: Set reference values

We set the reference grade to the one with the best gender balance. This ensures our baseline group has enough data from both genders for reliable estimates.

```
# Determine reference grade by minority count (best gender balance)
tab <- with(df, table(grade, gender))
minority_counts <- pmin(tab[, "Male"], tab[, "Female"])
ref_grade <- names(minority_counts)[which.max(minority_counts)]

# Set grade factor with chosen baseline
df$grade <- relevel(factor(df$grade), ref = ref_grade)

# Convert other categorical variables to factors
df$job_family <- as.factor(df$job_family)

# Verify reference levels
cat("Gender reference level:", levels(df$gender)[1], "\n")
cat("Grade reference level:", levels(df$grade)[1], "\n")
```

Step 5: Run the regression

With your reference levels in place, fit the log-pay model including the gender × grade interaction plus any user-specified controls. Make sure to replace our controls with your actual covariates, and be sure the column names in your data frame match

```
# Fit the linear model on log-adjusted pay
fit <- lm(log_pay ~ gender * grade + seniority + job_family, data = df)

# View results: coefficients, R-squared, p-values
summary(fit)

# Check model diagnostics
par(mfrow = c(2, 2))
plot(fit)
par(mfrow = c(1, 1))
```

Step 6: Extract Coefficients

Once the model is fit, we pull out the main gender effect (βFemale) and each grade-specific interaction. In R any non-estimable term will appear as 'NA'.

```
# Extract all coefficients into a named vector
coefs <- coef(fit)

# Main gender effect (reference grade)
beta_F <- coefs["genderFemale"]

# Identify all grade-specific interaction names
int_names <- grep("^genderFemale:grade", names(coefs), value = TRUE)

# Subset those interactions; R will give NA for any dropped terms
beta_ints <- coefs[int_names]
```

Step 7: Compute Grade-Level Pay Gap

We convert each log-scale coefficient into a percentage pay gap. Finally, we round to one decimal place and assemble a neat table.

```r
# Reference grade gap
gap_ref <- 1 - exp(beta_F)

# Other grades: apply interaction adjustment
gaps <- sapply(int_names, function(nm) {
  beta_int <- coefs[nm]
  if (is.na(beta_int)) {
    return(NA)
  }
  1 - exp(beta_F + beta_int)
})

# Extract grade names from interaction terms
grade_names <- gsub("^genderFemale:grade", "", int_names)

# Build results data frame
result <- data.frame(
  Grade = c(ref_grade, grade_names),
  PayGap_Percent = round(100 * c(gap_ref, gaps), 1),
  stringsAsFactors = FALSE
)

# Display results
cat("\n=== Grade-level Pay Gaps (%) ===\n")
print(result, row.names = FALSE)
```

4. 2. Measuring pay gaps using python

If you haven't got Python installed already, the first step is to download and install Python

- **Download Python:** https://www.python.org/

Step 1: Install and load packages

If you haven't got the packages needed for this analysis, the first step will be to install and load them.

```
pip install pandas numpy statsmodels
```

Load libraries:

```
import pandas as pd
import numpy as np
import patsy
import statsmodels.api as sm
```

Step 2: Load Data

The first step is to load in our dataset. A good practice is to visually review your data to make sure the data conforms to our expectations and there are no missing values.

```
# Load employee data
df = pd.read_csv("eu_company_pay_data.csv")

# View first few rows
print(df.head())

# Check structure (similar to R's str())
print(df.info())
```

Step 3: Prepare Variables

Before you start your pay equity analysis, make sure your data is adjusted for FTE, your compensation variable is log-transformed and the reference gender is set to male .

```
# Calculate full-time equivalent total pay
df["total_pay_fte"] = df["total_pay"] / df["fte"]

# Create log-transformed pay (for percentage interpretation)
df["log_pay"] = np.log(df["total_pay_fte"])
```

Step 4: Set reference values

We set the reference grade to the one with the best gender balance. This ensures our baseline group has enough data from both genders for reliable estimates.

```
# 1. Gender: define Male as base; no need for C(...) if only two levels
df['gender'] = pd.Categorical(df['gender'],
categories=['Male','Female'])
# 2. Compute minority counts to pick ref_grade
counts = df.groupby(['grade','gender']).size().unstack(fill_value=0)
minority = np.minimum(counts['Male'], counts['Female'])
ref_grade = minority.idxmax() # highest minority count

# 3. Reorder Grade so ref_grade comes first
all_grades = list(np.unique(df['grade']))
other_grades = [g for g in all_grades if g != ref_grade]
df['grade'] = pd.Categorical(df['grade'],
categories=[ref_grade] + other_grades)
# 4. Verify references
assert df['gender'].cat.categories[0] == 'Male'
assert df['grade'].cat.categories[0] == ref_grade
```

Step 5: Run the regression

With your reference levels in place, fit the log-pay model including the gender × grade interaction plus any user-specified controls. Make sure to replace our controls with your actual covariates, and be sure the column names in your data frame match

```
# 1. Build response (y) and design matrix (X)
y, X = patsy.dmatrices(
'log_pay ~ gender * grade + seniority + C(job_family)',
df,
return_type='dataframe'
)
# 2. Drop any columns that are all zeros (no data to estimate)
nonzero = X.columns[(X != 0).any(axis=0)]
X_trimmed = X[nonzero]
# 3. Fit the trimmed OLS model
fit = sm.OLS(y, X_trimmed).fit()
# 5. Inspect summary: coefficients correspond only to estimable terms
print(fit.summary())
```

Step 6: Extract Coefficients

Once the model is fit, we pull out the main gender effect (βFemale) and each grade-specific interaction. In R any non-estimable term will appear as 'NA'.

```
# Pull the fitted parameters into a Series
b = fit.params
# Main gender effect (reference grade)
beta_F = b['gender[T.Female]']
# For each non-reference grade, get the interaction term or NaN if absent
beta_ints = {
    g: b.get(f'gender[T.Female]:grade[T.{g}]', np.nan)
    for g in df['grade'].cat.categories
    if  g != ref_grade
}
```

Step 7: Compute Grade-Level Pay Gap

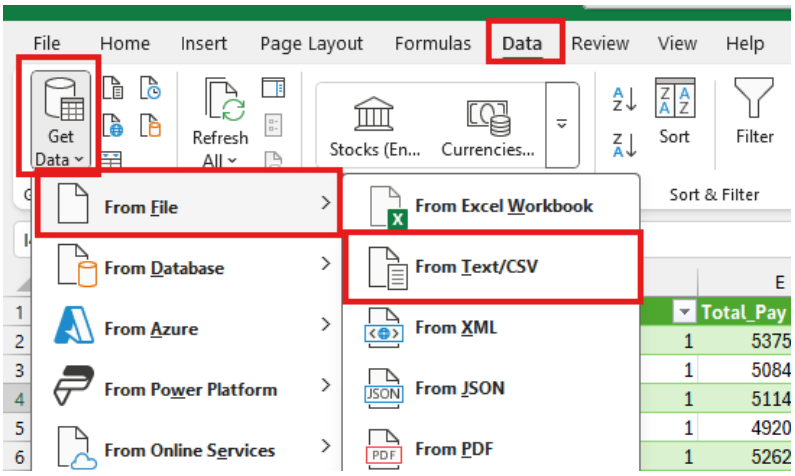
We convert each log-scale coefficient into a percentage pay gap. Finally, we round to one decimal place and assemble a neat table.

```
# 1. Reference grade gap
gap_ref = 1 - np.exp(beta_F)
rows = [(ref_grade, round(100 * gap_ref, 1))]

# 2. Other grades: include NaN where interaction was absent
for g, bi in beta_ints.items():
    gap = 1 - np.exp(beta_F + bi) # bi may be np.nan
    rows.append((g, round(100 * gap, 1)))

# 3. Assemble into a DataFrame
df_gaps = pd.DataFrame(rows, columns=['Grade','AdjustedPayGap'])

print(df_gaps)
```

4.3 Measuring pay gaps using excel

Step 1: Load Data

Go to the Data -> Get Data -> From File -> From Text/CSV



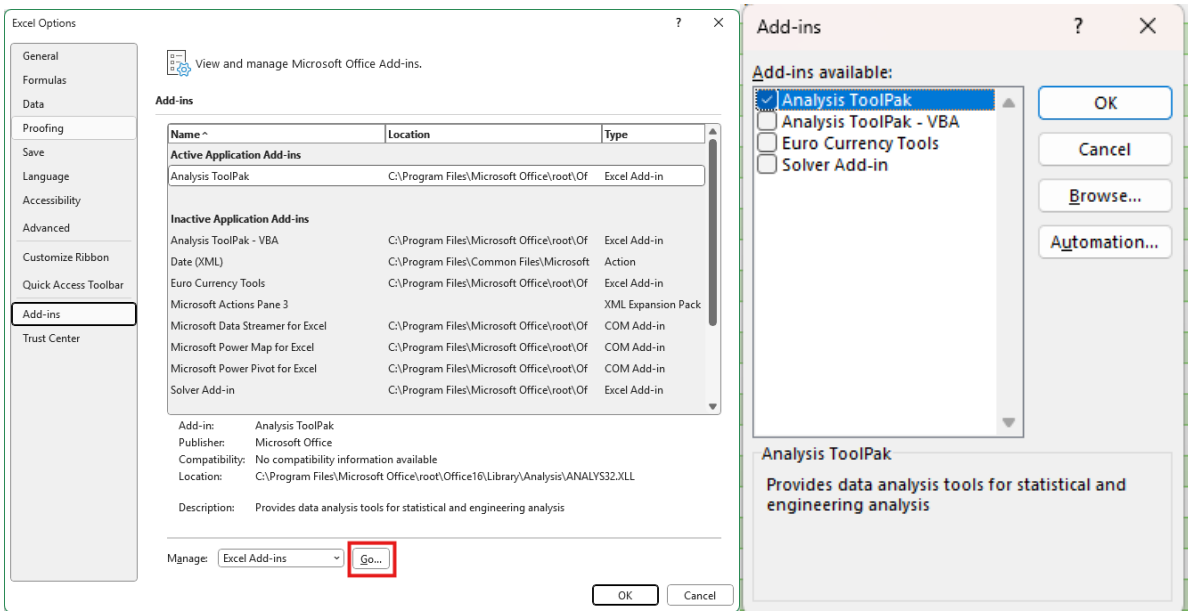
The dataset imported in this example is called "dataset" and is shown below.

	A	B	C	D	E	F	G
1	employee_id	gender	grade	fte	total_pay	seniority	job_family
2	W001	Male	2	1	27098	14	IT
3	W002	Male	3	1	13223	110	Admin
4	W003	Female	2	1	25301	115	HR
5	W004	Female	2	1	79470	79	IT
6	W005	Male	2	1	24632	36	HR
7	W006	Male	5	1	25413	62	Admin
8	W007	Male	2	1	9003	31	IT
9	W008	Male	1	1	25193	7	Admin
10	W009	Male	2	1	25898	88	IT
11	W010	Male	3	1	12289	67	IT
12	W011	Female	1	1	26100	49	HR
13	W012	Male	3	1	50267	118	IT

Step 2: Enable 'Analysis ToolPak'

To create the regression in Excel, 'Analysis ToolPak' is needed. To enable it go to:

1. File → Options → Add-Ins → Go. . . (at bottom).
2. Check "Analysis ToolPak" and click OK.



Step 3: Set Reference Levels

We set the reference grade to the one with the best gender balance. This ensures our baseline group has enough data from both genders for reliable estimates.

List all grades in A2:

```
=SORT(UNIQUE(dataset[Grade]))
```

Count men in each grade. Write in B2:

```
=COUNTIFS(dataset[Grade],A2#,dataset[Gender],"Male")
```

Count women in each grade. Write in C2:

```
=COUNTIFS(dataset[Grade],A2#,dataset[Gender],"Female")
```

Pick the minimum count for each grade. Write in D2:

```
=MAP(B2#,C2#,LAMBDA(a,b, MIN(a,b)))
```

Set the Reference Grade in E2:

```
=INDEX(A2#, MATCH(MAX(D2#), D2#, 0))
```

Add headers to get something like this:

	A	B	C	D	E
1	Grade	Male	Female	Min	Reference_Grade
2	1	60	53	53	2
3	2	102	73	73	
4	3	99	58	58	
5	4	30	15	15	
6	5	5	5	5	

Step 4: Create Model Matrix

'Analysis ToolPak' needs a model matrix to run regression. Create a new sheet called 'model_matrix'. Define the Log_Pay in A3:

```
=LN(dataset[Total_Pay]/dataset[FTE])
```

We transform gender to be 1 if Female and 0 if Male. Write in B3:

```
=IF(dataset[Gender]="Female",1,0)
```

For grades we create one column for each grade except the reference grade. Every column contains a 1 if the employee is in the grade and 0 if not. To add the grades to the model matrix, write in C2:

```
=TRANSPOSE(FILTER(
SORT(UNIQUE(dataset[Grade])),
SORT(UNIQUE(dataset[Grade])) <> reference_grade!E2
))
```

And then in C3:

```
=--(dataset[Grade] = C2#)
```

Categorical variables

To add categorical variables such as Job Family, we use a similar approach to grades. Write in G2 (Or the next available cell in row 2):

```
=TRANSPOSE(UNIQUE(dataset[Job_Family]))
```

And then in G3 (Or the next available cell in row 3):

```
=--(dataset[Job_Family]=G2#)
```

Numerical variables

To add numerical variables such as Seniority, write in the next available cell in row 3:

```
=dataset[Seniority]
```

Interaction variables

We create interaction columns between gender and grade to estimate the adjusted gender pay gap. In the next available cell in row 2 write:

```
=B2 & ":" & C2#
```

And in the cell below:

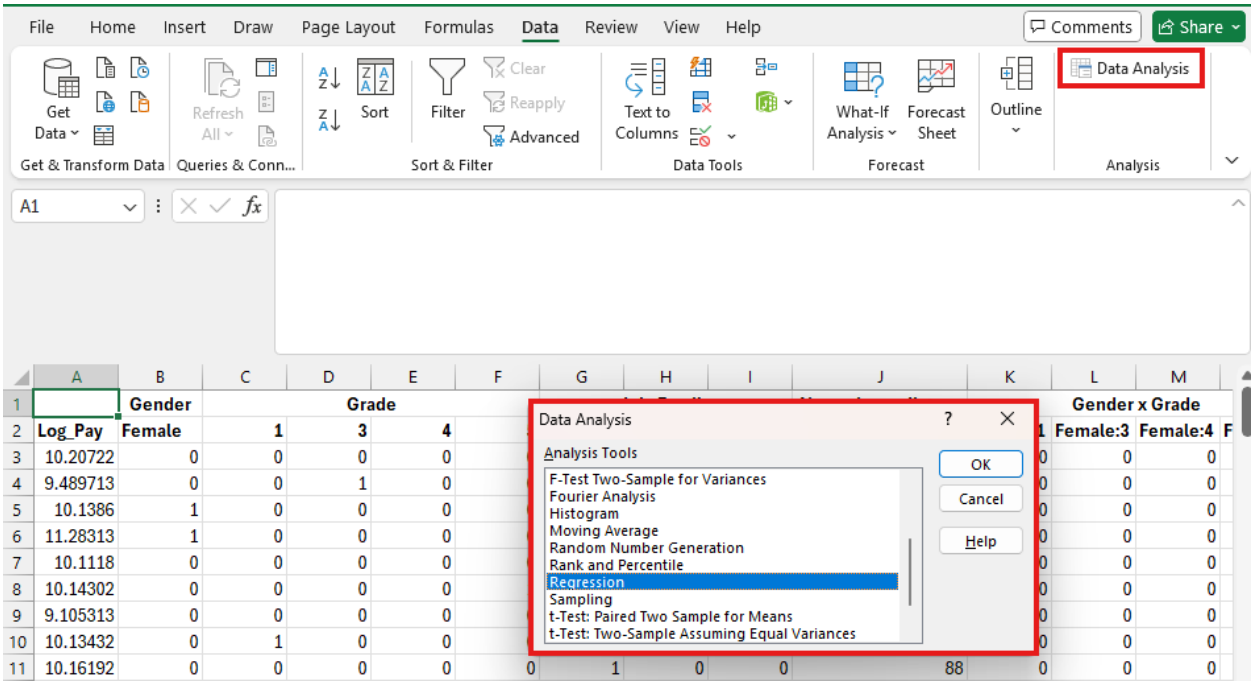
```
=B3# * C3#
```

Add labels to all columns and optional labels in row 1 for categorical variables:

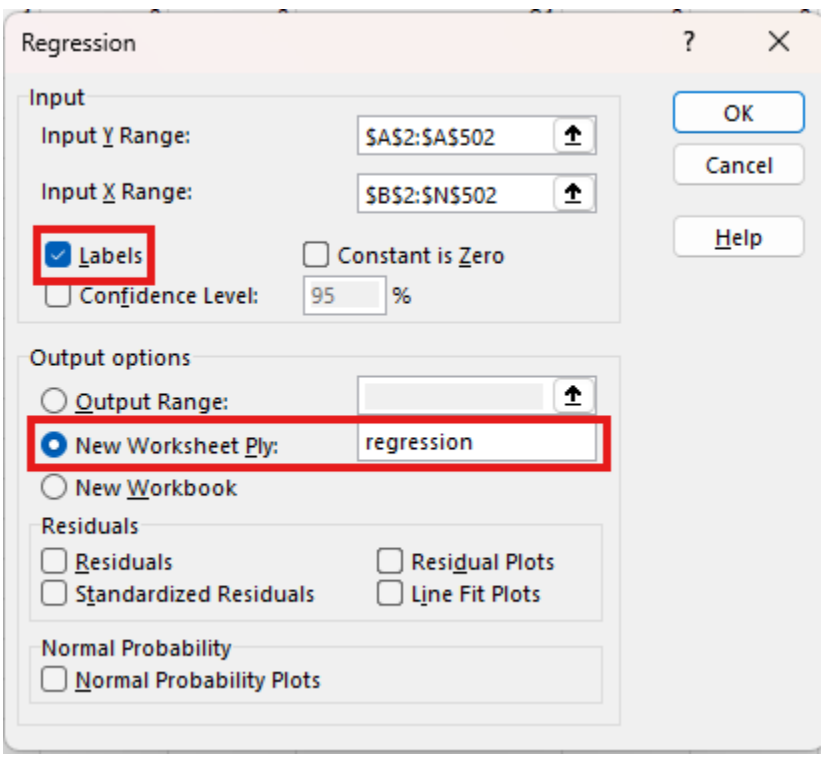
	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1		Gender	Grade				Job_Family			Numeric predictors	Gender x Grade			
2	Log_Pay	Female	1	3	4	5	IT	Admin	HR	Seniority	Female:1	Female:3	Female:4	Female:5
3	10.20722	0	0	0	0	0	1	0	0	14	0	0	0	0
4	9.489713	0	0	1	0	0	0	1	0	110	0	0	0	0
5	10.1386	1	0	0	0	0	0	0	1	115	0	0	0	0
6	11.28313	1	0	0	0	0	1	0	0	79	0	0	0	0
7	10.1118	0	0	0	0	0	0	0	1	36	0	0	0	0
8	10.14302	0	0	0	0	1	0	1	0	62	0	0	0	0
9	9.105313	0	0	0	0	0	1	0	0	31	0	0	0	0
10	10.13432	0	1	0	0	0	0	1	0	7	0	0	0	0
11	10.16192	0	0	0	0	0	1	0	0	88	0	0	0	0

Step 5: Run Regression

Go to Data -> Data Analysis -> Regression.



Set Log_Pay of the model_matrix as the Y Range and the remaining columns as the X Range. Include the labels in row 2. Toggle Labels and set the name of the New Worksheet to “regression”. Click OK.



This will automatically generate the “regression” sheet.

	A	B	C	D	E	F	G	H	I
1	SUMMARY OUTPUT								
2									
3	*Regression Statistics*								
4	Multiple R	0.081686961							
5	R Square	0.00667276							
6	Adjusted R Square	-0.019856864							
7	Standard Error	0.736002557							
8	Observations	500							
9									
10	ANOVA								
11		*df*	*SS*	*MS*	*F*	*Significance F*			
12	Regression	13	1.772151081	0.136319314	0.27262197	0.994957556			
13	Residual	487	263.8077848	0.541699764					
14	Total	500	265.5799359						
15									
16		*Coefficients*	*Standard Error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*	*Lower 95.0%*	*Upper 95.0%*
17	Intercept	10.22469797	0.106825214	95.71427543	0	10.01480276	10.43459318	10.01480276	10.43459318
18	Female	0.043424568	0.11330807	0.383243385	0.701706546	-0.179208465	0.266057601	-0.179208465	0.266057601
19	1	-0.020466358	0.12004012	-0.170495982	0.864690902	-0.256326839	0.215394123	-0.256326839	0.215394123
20	3	-0.042016479	0.104019485	-0.403928927	0.686442296	-0.246398863	0.162365905	-0.246398863	0.162365905
21	4	0.034068014	0.152913225	0.222793119	0.823789935	-0.266383092	0.334519121	-0.266383092	0.334519121
22	5	-0.028002861	0.338677619	-0.082682939	0.934137624	-0.693452598	0.637446877	-0.693452598	0.637446877
23	IT	0	0	65535	#NUM!	0	0	0	0
24	Admin	0.017896304	0.078264205	0.228665252	#NUM!	-0.135880892	0.1716735	-0.135880892	0.1716735
25	HR	0.036004537	0.085366836	0.421762581	0.673384429	-0.131728243	0.203737317	-0.131728243	0.203737317
26	Seniority	-0.000266689	0.000988251	-0.269859184	0.787382899	-0.002208451	0.001675074	-0.002208451	0.001675074
27	Female:1	0.016931869	0.179617031	0.094266501	0.924936231	-0.335988133	0.369851871	-0.335988133	0.369851871
28	Female:3	-0.106098821	0.166066648	-0.638893011	0.523192896	-0.432394393	0.220196751	-0.432394393	0.220196751
29	Female:4	-0.171766982	0.2597342	-0.661318308	0.508720949	-0.682104972	0.338571008	-0.682104972	0.338571008
30	Female:5	-0.32346937	0.479975082	-0.673929506	0.500675871	-1.266547019	0.619608279	-1.266547019	0.619608279

Step 6: Compute Grade-Level Pay Gaps

Finally, create a new sheet called “results”. Write “Grade” in A1 and “Adjusted Pay Gap” in B1. Write the reference level in A2 and the remaining grades just below. Write in B2:

```
=(1 - EXP( INDEX(regression!$B:$B, MATCH("Female", regression!$A:$A,0))))
```

Write in B3:

```
=(1 - EXP(
INDEX(regression!$B:$B, MATCH("Female", regression!$A:$A,0))
+ INDEX(regression!$B:$B, MATCH(
"Female:" & $A3, regression!$A:$A,0))
))
```

And drag down to calculate for all remaining grades. Format column B as percentages to get the final adjusted pay gap table.

	A	B
1	Grade	Adjusted Pay Gap
2	2	-4.4%
3	1	-6.2%
4	3	6.1%
5	4	12.0%
6	5	24.4%

5. How to close pay gaps

The goal of pay gap closure is not to equalize pay mechanically, but to correct **systematic underpayment** while preserving legitimate pay differences due to role, level, and experience and whatever else may be relevant. To do this in a transparent and defensible way, we rely on the same statistical framework that is used to measure the adjusted gender pay gap.

Using a single, consistent model ensures that:

- The definition of "underpaid" is objective and reproducible
- Pay adjustments align directly with how the pay gap is measured
- The process can be audited and explained to stakeholders

Rather than distributing uniform or discretionary raises, the approach prioritizes employees who are paid furthest below what the model predicts for comparable roles and characteristics.

To assess underpayment, we rely on the same pay model that is used to compute the adjusted gender pay gap:

$$\text{Pay} = \beta_0 + \beta_1 \text{Female} + \beta_2 \text{Grade} + \beta_3 \text{Seniority} + \varepsilon$$

For each worker:

- The **expected pay** is given by the fitted model
- The **residual** ε is the difference between actual and expected pay

$$\varepsilon = \text{Actual Pay} - \text{Expected Pay}$$

The residual provides a natural and consistent measure of relative underpayment:

- $\varepsilon < 0$: the worker is underpaid relative to peers with similar characteristics
- $\varepsilon > 0$: the worker is overpaid relative to the model

Residuals therefore allow employees to be ranked by underpayment in a way that is independent of grade, seniority, or gender.

To close the pay gap, we raise compensation for the most underpaid workers first. This is done by defining a **threshold** γ:

- Only workers of the underpaid gender receives raises
- Workers with residuals below γreceive a raise
- Workers with residuals at or above γreceive no adjustment

Equivalently, for workers belonging to the underpaid gender, pay is increased by:

$$\text{raise} = \max(0, \gamma - \varepsilon)$$

This approach raises the "bottom line" of compensation, ensuring that the most underpaid workers receive the largest adjustments, while leaving others unchanged.

Once raises are applied, the adjusted gender pay gap is recalculated using the same model definition as before.

The value of γ directly controls the extent of gap closure:

- **Higher** $\gamma \rightarrow$ more workers receive raises and raises are larger $\rightarrow$ the pay gap closes more
- **Lower** $\gamma \rightarrow$ fewer and smaller raises $\rightarrow$ the pay gap closes less

The threshold γ is identified using a **binary search** procedure:

1. Start with an interval of plausible threshold values
2. Guess a value of γ and apply the corresponding raises
3. Re-calculate the adjusted pay gap
4. Update the search interval:
    - If the gap is **smaller than the target** (overshoot), lower γ
    - If the gap is **larger than the target** (undershoot), increase γ
5. Repeat until the adjusted pay gap matches the target within a chosen tolerance

When closing pay gaps at the **grade level**, the same method is applied separately within each grade:

- Residuals are evaluated relative to employees in the same grade
- A distinct threshold γ_g is computed for each grade g
- Raises are applied only within that grade using its own threshold

This ensures that pay adjustments respect organizational structure and that underpayment is addressed among comparable roles.

In summary, pay gap closure is achieved by:

- Measuring underpayment using model residuals
- Raising the lowest residuals to a common threshold γ
- Selecting γ via binary search so the adjusted pay gap matches a predefined target

This method is transparent, data-driven, and directly aligned with the adjusted pay gap methodology used for reporting.

5.1. Close Pay Gaps using R

Step 1: Load data

The first step is to load in our dataset. A good practice is to visually review your data to make sure the data conforms to our expectations and there are no missing values.

```
df <- read.csv('../data/eu_company_pay_data.csv')

# Check structure
str(df)

# View first few rows
head(df)
```

Step 2: Prepare variables

Before you start your pay equity analysis, make sure your data is adjusted for FTE, your compensation variable is log-transformed and the reference gender is set to male.

```
# Calculate full-time equivalent total pay
df$total_pay_fte <- df$total_pay / df$fte

# Create log-transformed pay (for percentage interpretation)
df$log_pay <- log(df$total_pay_fte)

# Set Male as reference level for gender
df$gender <- relevel(factor(df$gender), ref = "Male")
```

Step 3: Set reference values

We set the reference grade to the one with the best gender balance. This ensures our baseline group has enough data from both genders for reliable estimates.

```
# Determine reference grade by minority count (best gender balance)
tab <- with(df, table(grade, gender))
minority_counts <- pmin(tab[, "Male"], tab[, "Female"])
ref_grade <- names(minority_counts)[which.max(minority_counts)]

# Set grade factor with chosen baseline
```

```
df$grade <- relevel(factor(df$grade), ref = ref_grade)

# Convert other categorical variables to factors
df$job_family <- as.factor(df$job_family)

# Verify reference levels
cat("Gender reference level:", levels(df$gender)[1], "\n")
cat("Grade reference level:", levels(df$grade)[1], "\n")
```

Step 4: Define gender-pay-gap function

Define a function to calculate the adjusted pay gap.

```
calculate_pay_gap <- function(fit) {
  # Extract all coefficients into a named vector
  coefs <- coef(fit)

  # Main gender effect (reference grade)
  beta_F <- coefs["genderFemale"]

  # Identify all grade-specific interaction names
  int_names <- grep("^genderFemale:grade", names(coefs), value = TRUE)

  # Subset those interactions; R will give NA for any dropped terms
  beta_ints <- coefs[int_names]

  # Reference grade gap
  gap_ref <- 1 - exp(beta_F)

  # Other grades: apply interaction adjustment
  gaps <- sapply(int_names, function(nm) {
    beta_int <- coefs[nm]
    if (is.na(beta_int)) {
      return(NA)
    }
    1 - exp(beta_F + beta_int)
  })

  # Extract grade names from interaction terms
  grade_names <- gsub("^genderFemale:grade", "", int_names)
```

```
  # Build results data frame
  result <- data.frame(
    Grade = c(ref_grade, grade_names),
    PayGap_Percent = round(100 * c(gap_ref, gaps), 1),
    stringsAsFactors = FALSE
  )

  return(result)
}
```

Step 5: Set initial values

Set the initial values before closing the pay gap.

- target_gap: The target pay gap (0.05 = 5%)
- gmin/gmax: The minimum and maximum values of the threshold
- gamma: The initial guess of the threshold gamma
- targets: Target gap set to be in the same "direction" as the pay gap. If the current pay gap is -10% then the target gap is -5% for that grade.

```
target_gap <- 5 # Set this to your target pay gap (5%)
fit <- lm(log_pay ~ gender * grade + seniority + job_family, data = df)
initial_pay_gap <- calculate_pay_gap(fit)
residuals <- fit$residuals

gmin <- rep(-10, nrow(initial_pay_gap))
gmax <- rep(10, nrow(initial_pay_gap))
gamma <- gmin
targets <- sign(initial_pay_gap$PayGap_Percent) * target_gap
```

Step 6: Close pay gaps

Run the binary search algorithm to determine the threshold gamma and apply raises.

```
for (j in 1:100) {
  df$new_log_pay <- df$log_pay
  # Raise all workers in scope to threshold
  for (i in 1:nrow(initial_pay_gap)) {
    gender_in_scope <- if (initial_pay_gap$PayGap_Percent[i] > 0) {
      "Female"
    } else "Male"
    idx <- df$grade == initial_pay_gap$Grade[i] &
```

```
            df$gender == gender_in_scope
    df$new_log_pay[idx] <- df$new_log_pay[idx] +
        pmax(residuals[idx], gamma[i]) - residuals[idx]
  }
  # Calculate new pay gap
  new_fit <- lm(
    new_log_pay ~ gender * grade + seniority + job_family,
    data = df
  )
  new_pay_gap <- calculate_pay_gap(new_fit)
  distance <- sign(initial_pay_gap$PayGap_Percent) *
              (new_pay_gap$PayGap_Percent - targets)

  # Update gamma
  overshot <- distance < 0
  undershot <- distance > 0
  gmax[overshot] <- gamma[overshot]
  gmin[undershot] <- gamma[undershot]

  gamma <- (gmin + gmax) / 2
}
```

Step 7: Evaluate results

The pay gaps should now be closed. Run this code to see the cost to close, the number of raises, and the new pay gap.

```
df$raise <- round(exp(df$new_log_pay) - df$total_pay_fte, 2)
df$new_total_pay_fte <- df$total_pay_fte + df$raise

# Cost to close
sum(df$raise * df$fte)

# Number of raises
sum(df$raise>0)

# New pay gap
new_pay_gap
```

5.2 Close Pay Gaps using python

Step 1: Load data and libraries

The first step is to load in our dataset.

```
import pandas as pd
import numpy as np
import patsy
import statsmodels.api as sm

df = pd.read_csv("../data/eu_company_pay_data.csv")
df
```

Step 2: Prepare variables

Before you start your pay equity analysis, make sure your data is adjusted for FTE, your compensation variable is log-transformed and the reference gender is set to male.

```
# Calculate full-time equivalent total pay
df["total_pay_fte"] = df["total_pay"] / df["fte"]

# Create log-transformed pay (for percentage interpretation)
df["log_pay"] = np.log(df["total_pay_fte"])

# Set "Male" as reference gender
df['gender'] = pd.Categorical(df['gender'],
categories=['Male','Female'])
```

Step 3: Set reference values

We set the reference grade to the one with the best gender balance. This ensures our baseline group has enough data from both genders for reliable estimates.

```
# 1. Compute minority counts to pick ref_grade
counts = df.groupby(['grade','gender']).size().unstack(fill_value=0)
minority = np.minimum(counts['Male'], counts['Female'])
ref_grade = minority.idxmax() # highest minority count

# 2. Reorder Grade so ref_grade comes first
all_grades = list(np.unique(df['grade']))
```

```
other_grades = [g for g in all_grades if g != ref_grade]
df['grade'] = pd.Categorical(df['grade'],
categories=[ref_grade] + other_grades)

# 3. Verify references
assert df['gender'].cat.categories[0] == 'Male'
assert df['grade'].cat.categories[0] == ref_grade
```

Step 4: Define model

Define the pay model. This is the basis of calculating adjusted pay gaps and expected pay.

```
# 1. Build response (y) and design matrix (X)
y, X = patsy.dmatrices(
'log_pay ~ gender * grade + seniority + C(job_family)',
df,
return_type='dataframe'
)
# 2. Drop any columns that are all zeros (no data to estimate)
nonzero = X.columns[(X != 0).any(axis=0)]
X_trimmed = X[nonzero]
```

Step 5: Define gender-pay-gap function

Define a function to calculate the adjusted pay gap.

```
def calculate_pay_gap(fit):
    # Pull the fitted parameters into a Series
    b = fit.params
    # Main gender effect (reference grade)
    beta_F = b['gender[T.Female]']
    # For each non-reference grade
    # get the interaction term or NaN if absent
    beta_ints = {
        g: b.get(f'gender[T.Female]:grade[T.{g}]', np.nan)
        for g in df['grade'].cat.categories
        if  g != ref_grade
    }

    # 1. Reference grade gap
```

```
    gap_ref = 1 - np.exp(beta_F)
    rows = [(ref_grade, round(100 * gap_ref, 1))]

    # 2. Other grades: include NaN where interaction was absent
    for g, bi in beta_ints.items():
        gap = 1 - np.exp(beta_F + bi) # bi may be np.nan
        rows.append((g, round(100 * gap, 1)))

    # 3. Assemble into a DataFrame
    df_gaps = pd.DataFrame(rows, columns=['Grade','AdjustedPayGap'])

    return df_gaps
```

Step 6: Set initial values

Set the initial values before closing the pay gap.

- target_gap: The target pay gap (0.05 = 5%)
- gmin/gmax: The minimum and maximum values of the threshold
- gamma: The initial guess of the threshold gamma
- targets: Target gap set to be in the same "direction" as the pay gap. If the current pay gap is -10% then the target gap is -5% for that grade.

```
target_gap = 5 # Set this to target pay gap (5%)
fit = sm.OLS(y, X_trimmed).fit()
initial_pay_gap = calculate_pay_gap(fit)
residuals = fit.resid

gmin = np.array([-10.] * len(initial_pay_gap))
gmax = np.array([10.] * len(initial_pay_gap))
gamma = gmin
targets = np.sign(initial_pay_gap["AdjustedPayGap"]) * target_gap
```

Step 7: Close pay gaps

Run the binary search algorithm to determine the threshold gamma and apply raises.

```
for j in range(100):
    # Initialize new log pay
    df["new_log_pay"] = df["log_pay"]
    # Raise all workers in scope up to threshold
    for i in range(len(initial_pay_gap)):
        gender_in_scope = "Female" if initial_pay_gap.loc[i,
"AdjustedPayGap"] > 0 else "Male"
        idx = np.logical_and(df["grade"] == initial_pay_gap.loc[i,
"Grade"], df["gender"] == gender_in_scope)
        df.loc[idx, "new_log_pay"] += np.maximum(residuals[idx], gamma[i])
- residuals[idx]

    # Calculate new pay gap
    new_fit = sm.OLS(df["new_log_pay"], X_trimmed).fit()
    new_pay_gap = calculate_pay_gap(new_fit)

    # Update threshold
    distance = np.sign(initial_pay_gap["AdjustedPayGap"]) *
(new_pay_gap["AdjustedPayGap"] - targets)

    overshot = np.array(distance < 0)
    undershot = np.array(distance > 0)
    gmax[overshot] = gamma[overshot]
    gmin[undershot] = gamma[undershot]

    gamma = (gmax + gmin) / 2
```

Step 7: Evaluate results

The pay gaps should now be closed. Run this code to see the cost to close, the number of raises, and the new pay gap.

```
df["raise"] = np.round(np.exp(df["new_log_pay"]) - df["total_pay_fte"], 2)
df["new_total_pay_fte"] = df["total_pay_fte"] + df["raise"]

# Cost to close
print(f"Total cost: {sum(df["raise"] * df["fte"])}")
```

```
# Number of raises
print(f"Number of raises: {sum(df["raise"]>0)}")

# New pay gap
new_pay_gap
```